Software Requirements Specification

for

Integrated Fitness & Resource Management System (IFRMS)

Version 1.0

Prepared by:

Group Name: CS03_4

Name	Roll No	Email
N.Jatin	B240250CS	nalli_b240250cs@nitc.ac.in
K.D.Karun Lakshman	B240213CS	kodeti_b240213cs@nitc.ac.in
A.N.S.Thanush Reddy	B2402133CS	ambati_b240133cs@nitc.ac.in
J.Manish Reddy	B240200CS	jakka_b240200cs@nitc.ac.in

Instructor: Dr. A. Sudarshan Chakravarthy

Course: DBMS (CS2011E)

Date: 14/03/2026

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	CS03_04	Initial SRS Documentation for IFRMS	13/03/2026

Contents

1. Introduction ...4

1.1 Document Purpose
1.2 Product Scope
1.3 Intended Audience
1.4 Definitions, Acronyms and Abbreviations

2. Overall Description ...5

2.1 Product Overview
2.2 Product Functionality
2.3 Design and Implementation Constraints

3. Specific Requirements ...6

3.1 External Interface Requirements ...
3.2 Functional Requirements ...
3.1 Use Case Model ...
3.3.1 Use Case #1: Register Member (U1) ...
3.3.2 Use Case #2: Enroll in Fitness Program (U2) ...
3.3.3 Use Case #3: Create Workout Plan (U3) ...
3.3.4 Use Case #4: Track Member Progress (U4) ...
3.3.5 Use Case #5: Manage Equipment (U5) ...
3.3.6 Use Case #6: Schedule Training Session (U6) ...
3.3.7 Use Case #7: Generate Reports (U7) ...

4. Other Non-functional Requirements ...13

4.1 Performance Requirements ...
4.2 Safety and Security Requirements ...
4.3 Software Quality Attributes ...
4.3.1 Performance ...
4.3.2 Maintainability and Scalability ...

5. Appendix A – Data Dictionary ...15

6. Appendix B – Group Log ...16

1 Introduction

This document specifies the software requirements for the Integrated Fitness & Resource Management System (IFRMS).

1.1 Document Purpose

The purpose of this document is to outline the functional and non-functional requirements for the IFRMS. It serves as a guide for developers and a reference for stakeholders to ensure all necessary features—from workout tracking to equipment management—are included.

1.2 Product Scope

The IFRMS is a web-based platform designed to centralize gym operations. It enables members to track workouts, book trainers, and manage memberships, while allowing administrators to monitor equipment health, schedule resources, and analyze facility usage. The system will be built using a modern stack: Next.js (frontend), Node.js/Express (backend), and PostgreSQL (database).

1.3 Intended Audience

This document is intended for project managers, frontend/backend developers and the project instructor.

1.4 Definitions, Acronyms and Abbreviations

- **IFRMS**: Integrated Fitness & Resource Management System
- **CRUD**: Create, Read, Update, Delete
- **JWT**: JSON Web Token
- **CRM**: Customer Relationship Management
- **DBMS**: Database Management System

2 Overall Description

2.1 Product Overview

IFRMS replaces manual gym management and fragmented tracking apps. It bridges the gap between member engagement and resource maintenance. The system handles real-time booking, automated equipment status alerts, and personalized fitness data storage.

2.2 Product Functionality

Member Functions:

- Log workout progress and set fitness goals.
- Book slots for facility resources (e.g., treadmills, yoga rooms).
- View and pay membership invoices.

Admin Functions:

- Manage inventory and log maintenance schedules for gym equipment.
- Assign personal trainers to members.
- Generate reports on peak facility usage hours.

2.3 Design and Implementation Constraints

- System must be accessible via standard web browsers (Chrome, Firefox, Safari).
- Database must ensure data integrity for concurrent booking sessions.
- Backend must follow RESTful API standards.

3 Specific Requirements

3.1 External Interface Requirements

User Interfaces: Responsive web dashboard for both desktop and mobile views. Focus on data visualization (charts) for fitness progress.

Software Interfaces: Integration with payment gateways (Stripe/PayPal Mock) and email services for notification alerts.

3.2 Functional Requirements

- **F1:** The system shall allow users to register and log in using secure password hashing and authentication mechanisms.

- **F2:** The system shall manage user roles (member, trainer, admin) and assign permissions based on the user type.

- **F3:** The system shall allow member registration and track membership details including member ID and membership date.

- **F4:** The system shall allow users to view and update profile information including name, email, phone number, and date of birth.

- **F5:** The system shall allow members to view and enroll in available fitness programs.

- **F6:** The system shall manage workout plans and training schedules including workout descriptions and session dates.

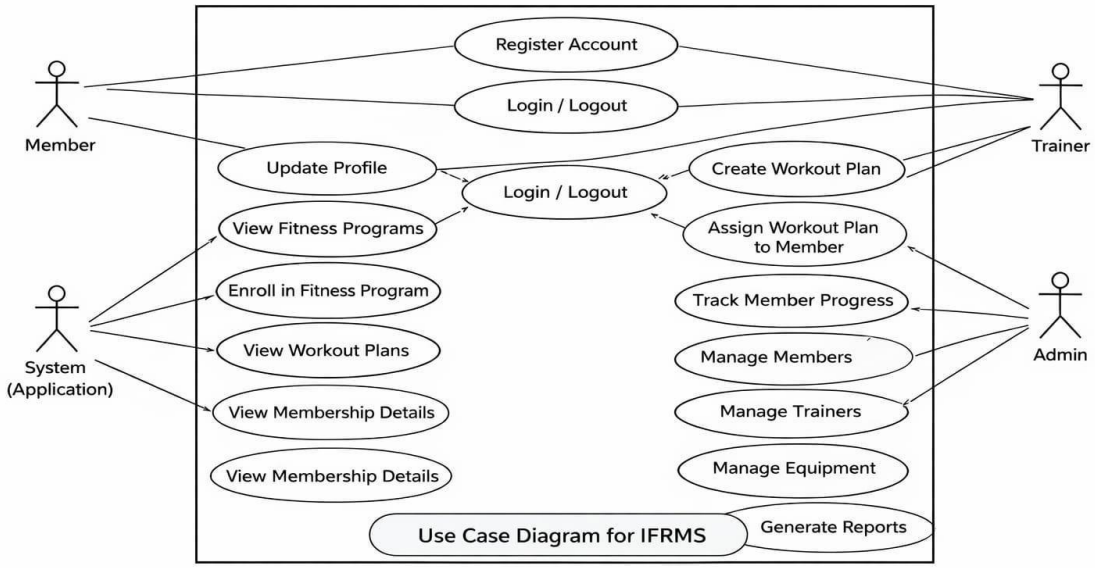
- **F7:** The system shall record and track member fitness progress including metrics such as weight, repetitions, and workout time.

- **F8:** The system shall manage trainer information including trainer ID and specialization areas.

- **F9:** The system shall track gym equipment, including equipment name and status (Available, In Use, Maintenance).

- **F10:** The system shall generate reports related to gym activities, member progress, and system usage.

3.3 Use Case Model



Use Case Diagram for IFRMS

3.3.1 Use Case #1: Register Member (U1)

● Author: Project Team
● Purpose: Allow a new member to register in the fitness management system.
● Requirements Traceability: The system should allow new users to create a member account and store their details in the database.
● Priority: High

● **Preconditions:**
○ The system must be running.
○ The user must provide valid registration details.

● **Postconditions:**
○ Member account is successfully created.
○ Member information is stored in the database.

● **Actors:**
○ Member (Primary)
○ Application (Processes and stores member data)

● **Basic Flow:**
○ Member opens the registration page.
○ Member enters personal details (name, email, phone number, password).
○ Member submits the registration form.
○ Application validates the information.
○ Application stores the member details in the database.
○ A confirmation message is displayed.

● **Exceptions:**
○ If the email already exists, registration is prevented.
○ If the server is down, registration cannot be completed.

3.3.2 Use Case #2: Enroll in Fitness Program (U2)

● Author: Project Team
● Purpose: Allow members to view available fitness programs and enroll in them.
● Requirements Traceability: Members should be able to view available programs and enroll in desired programs.
● Priority: High

● **Preconditions:**
○ The member must be logged in.
○ Fitness programs must be available in the system.

● **Postconditions:**
○ Member is successfully enrolled in the selected program.
○ Enrollment details are stored in the database.

● **Actors:**
○ Member (Primary)
○ Application (Processes and stores enrollment details)

● **Basic Flow:**
○ Member logs into the system.
○ Member navigates to the programs page.
○ Application displays available fitness programs.
○ Member selects the desired program.
○ Member submits the enrollment request.
○ Application stores enrollment details in the database.
○ A confirmation message is displayed.

● **Exceptions:**
○ If the program is full, enrollment is not allowed.
○ If the server is down, enrollment cannot be completed.

3.3.3 Use Case #3: Create Workout Plan (U3)

● Author: Project Team
● Purpose: Allow trainers to create workout plans for members.
● Requirements Traceability: Trainers should be able to create and assign workout plans for members.
● Priority: High

● **Preconditions:**
○ Trainer must be logged in.
○ Members must be registered in the system.

● **Postconditions:**
○ Workout plan is successfully created and stored.
○ Members can view the assigned workout plan.

● **Actors:**
○ Trainer (Primary)
○ Application (Processes and stores workout plans)

● **Basic Flow:**
○ Trainer logs into the system.
○ Trainer navigates to workout plan management page.
○ Trainer selects a member.
○ Trainer enters workout plan details.
○ Trainer submits the workout plan.
○ Application stores workout plan in the database.
○ A confirmation message is displayed.

● **Exceptions:**
○ If workout details are incomplete, system requests corrections.
○ If the server is down, workout plan cannot be saved.

3.3.4 Use Case #4: Track Member Progress (U4)

● Author: Project Team
● Purpose: Allow trainers to update and monitor the progress of members.
● Requirements Traceability: Trainers should be able to record fitness progress of members.
● Priority: High

● **Preconditions:**
○ Trainer must be logged in.
○ Member must exist in the system.

● **Postconditions:**
○ Progress records are successfully stored in the database.

● **Actors:**
○ Trainer (Primary)
○ Application (Processes and stores progress records)

● **Basic Flow:**
○ Trainer logs into the system.
○ Trainer navigates to member progress page.
○ Trainer selects a member.
○ Trainer enters progress data (weight, performance, etc.).
○ Application stores progress details in the database.
○ A confirmation message is displayed.

● **Exceptions:**
○ If invalid progress data is entered, system requests correction.

3.3.5 Use Case #5: Manage Equipment (U5)

● Author: Project Team
● Purpose: Allow administrators to manage gym equipment.
● Requirements Traceability: Admin should be able to add, update, or remove equipment.
● Priority: High

● **Preconditions:**
○ Admin must be logged in.

● **Postconditions:**
○ Equipment records are updated in the database.

● **Actors:**
○ Admin (Primary)
○ Application (Processes equipment management operations)

● **Basic Flow:**
○ Admin logs into the system.
○ Admin navigates to equipment management page.
○ Admin selects action (add/update/delete equipment).
○ Admin enters equipment details.
○ Application updates the database.
○ Confirmation message is displayed.

● **Exceptions:**
○ If required details are missing, the system prompts for correction.

3.3.6 Use Case #6: Schedule Training Session (U6)

● Author: Project Team
● Purpose: Allow trainers to schedule training sessions for members.
● Requirements Traceability: Trainers should be able to create and manage training schedules.
● Priority: High

● **Preconditions:**
○ Trainer must be logged in.
○ Members must exist in the system.

● **Postconditions:**
○ Training session schedule is stored in the database.

● **Actors:**
○ Trainer (Primary)
○ Application (Processes scheduling operations)

● **Basic Flow:**
○ Trainer logs into the system.
○ Trainer navigates to schedule page.
○ Trainer selects date and time for session.
○ Trainer assigns session to members.
○ Application stores schedule information in database.
○ Confirmation message is displayed.

● **Exceptions:**
○ If the session time conflicts with another session, system prevents scheduling.

3.3.7 Use Case #7: Generate Reports (U7)

● Author: Project Team
● Purpose: Allow administrators to generate reports about members, trainers, and equipment usage.
● Requirements Traceability: Admin should be able to generate system reports for monitoring and management.
● Priority: High

● **Preconditions:**
○ Admin must be logged in.

● **Postconditions:**
○ Reports are generated and displayed.

● **Actors:**
○ Admin (Primary)
○ Application (Processes report generation)

● **Basic Flow:**
○ Admin logs into the system.
○ Admin navigates to reports section.
○ Admin selects report type.
○ Application retrieves data from database.
○ System generates the report.
○ Report is displayed to the admin.

● **Exceptions:**
○ If there is no data available, system displays an appropriate message.

4 Other Non-functional Requirements

4.1 Performance Requirements

The system should respond quickly when users perform actions such as login, viewing workout plans, or checking schedules.

The application should handle multiple users (members, trainers, and admins) at the same time without slowing down.

Data such as workout plans, equipment details, and member information should load within a few seconds.

The system should store and retrieve data efficiently from the database.

4.2 Safety and Security Requirements

User information such as passwords and personal details must be stored securely in the database.

Only authorized users should be able to access the system through login authentication.

Different users (Admin, Trainer, Member) should only access the features allowed for their role.

The system should prevent unauthorized access and protect user data from misuse.

4.3 Software Quality Attributes

4.3.1 Performance

The system should work smoothly even when many users access it at the same time.

Database queries should be optimized to reduce waiting time for users.

The user interface should be simple and responsive so users can easily navigate the system.

4.3.2 Maintainability and Scalability

The system should be designed in a way that makes it easy to update or fix in the future.

Clear documentation should be available to help developers understand the system.

The system should allow adding new features such as new fitness programs or additional equipment without major changes.

A basic user guide should be provided to help users understand how to use the system.

Appendix A – Data Dictionary

Attribute	Description	Related Operations	Related Requirements
user_id	Unique ID (Member ID, Trainer ID, Admin ID)	Authentication, permissions	F1, F2
user_type	ENUM (member, trainer, admin)	User role management	F2
password	Hashed password storage	Login, authentication	F1
created_at	Timestamp for account creation	Account tracking	-
name	Full name	Display, profile update	F4
email	Contact information	Updates	F4
phone_number	Contact number	Updates	F4
dob	Date of birth	Profile information	F4
membership_date	Date when member joined gym	Membership tracking	F3
trainer_id	Unique trainer identifier	Trainer management	F8
specialization	Trainer specialization (Fitness, Yoga, Cardio)	Trainer profile management	F8
member_id	Unique member identifier	Member registration, tracking	F3
program_id	Unique fitness program identifier	Program enrollment	F5
program_name	Name of fitness program	Display, selection	F5
equipment_id	Unique equipment identifier	Equipment tracking	F9
equipment_name	Name of gym equipment	Resource management	F9
equipment_status	ENUM (Available, In Use, Maintenance)	Equipment monitoring	F9
workout_plan_id	Unique workout plan identifier	Workout management	F6
workout_description	Description of workout exercises	Workout plan creation	F6
progress_id	Unique progress record ID	Fitness progress tracking	F7
progress_value	Recorded fitness metrics (weight, reps, time)	Progress updates	F7
schedule_id	Unique schedule identifier	Training session scheduling	F6
session_date	Date of training session	Schedule management	F6
report_id	Unique report identifier	Report generation	F10

Appendix B – Group Log

The group worked together to develop the **Integrated Fitness & Resource Management System (IFRMS)** SRS document.

Initially, the team discussed the system idea and identified the main features such as member management, trainer scheduling, workout tracking, and equipment monitoring.

The work was divided among team members, where each person contributed to different sections such as functional requirements, use cases, and documentation formatting.

Regular discussions were held to review the progress and ensure that all sections of the document were consistent and complete.

The final document was compiled after verifying that all requirements and system details were clearly described.